

# **RUN, HIDE, ACTIVATE**

**A Design Project Report**

**Presented to the School of Electrical and Computer Engineering of Cornell University**

**in Partial Fulfillment of the Requirements of the Degree of**

**Master of Engineering, Electrical and Computer Engineering**

**Submitted by**

**Ashley Heckman (agh93)**

**Meng Field Advisor: Joe Skovira**

**Degree Date: December 2024**

## **Abstract**

**Master of Engineering Program**

**School of Electrical and Computer Engineering**

**Cornell University**

**Design Project Report**

**Project Title:** Run, Hide, Activate

**Author:** Ashley Heckman (agh93)

### **Abstract:**

The Run, Hide, Activate project is a system that provides an immediate response to active shooter situations. While police are trained to ignore distractions, most active shooters are not. The Run, Hide, Activate system takes advantage of this weakness by deploying nonlethal distractions around the shooter. It is comprised of multiple activation fobs, multiple distraction devices, and a central server. The distraction devices are equipped with a multitude of distractions that are randomly selected and deployed upon shooter detection. The distraction devices also communicate with the central server to provide an updated shooter location. Altogether, this system gives students extra time to get away and provides law enforcement critical information about the shooter's current whereabouts.

## **Executive Summary**

The Run, Hide, Activate system addresses the critical issue of active shooter incidents through an innovative response system. This project has reached many milestones. Most notably, there are two fully functional distraction devices, a fully functional central server, and a fully functional activation mechanism.

The distraction devices each have three distractions: flashing lights, playing loud noises, and shooting marbles. They send real-time information back to the central server, allowing administrators and law enforcement to track the shooter through the building.

The central server is the communication hub and keeps track of system information. It stores system status, log files from the distraction devices, and a map showing where the distraction devices are and whether or not they detect a shooter.

Communication between all devices is seamless. All devices communicate over MQTT with a QoS of one, meaning that the message is guaranteed to be received. Additionally, since MQTT is designed for areas with bad networks, it should work in nearly every situation.

## **Introduction**

A recurring issue with active shooter situations is that they unfold quickly. A lot of damage can be done before even the fastest law enforcement agencies arrive on scene. With the rising number of active shooter incidents in schools, it is essential that a robust and effective response system is developed to address the threats before police arrive. One system to help with these situations is the Run, Hide, Activate project. While law enforcement is trained to ignore distractions, most of these active shooters are not. If we can deploy some kind of nonlethal distractions around the shooters, we could save lives, prevent serious injuries, and even increase the chances of a successful intervention.

## **Specific Issues**

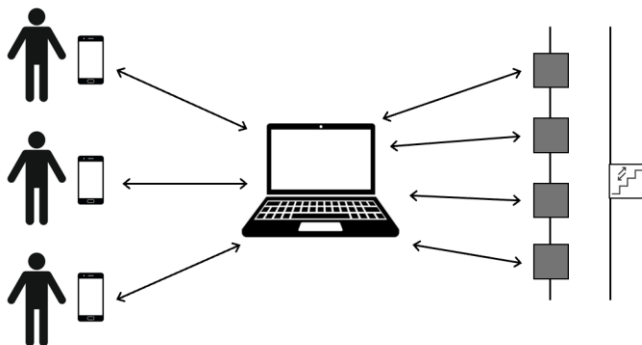
There are two specific points that highlight the need for the Run, Hide, Activate system. Firstly, most of the damage occurs within the first five to ten minutes of an active shooter incident. In fact, active shooters are frequently apprehended within ten minutes. This time between the start of the shooting and the arrival of law enforcement is critical for survival. Secondly, certain buildings present challenges for evacuation due to narrow hallways and infrequent exits. It might be impossible for students and teachers to run and leave the building. This calls for an immediate response system that can distract the shooter and buy people more time to run or hide.

## Range of Solutions

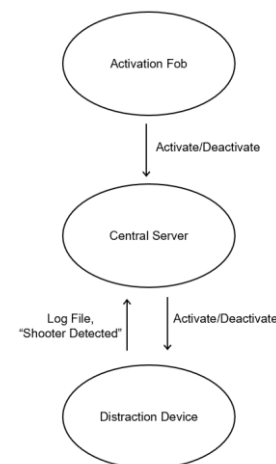
Creating a system to immediately deal with an active shooter is challenging. Since there is not a 100% guarantee that the system will only target the shooter, the system is severely limited in what it can do. The selected actions must hinder the shooter, but they cannot be dangerous. They also should not have a negative impact on people trying to flee. Given these constraints, there is not much that can be done to stop the shooter in their tracks. The best solution is to deploy a wide variety of nonlethal distractions around the shooter. These distractions are designed to throw off the shooter's accuracy without having a tangible impact on anyone else.

## Approach and Implementation

The Run, Hide, Activate system is composed of multiple activation fobs, a central server, and multiple distraction devices. The fob is used by staff to send an activation signal to the central server, which in turn sends an activation signal to the distraction devices. The distraction devices will then deploy upon detecting the shooter, simultaneously sending data back to the central server.



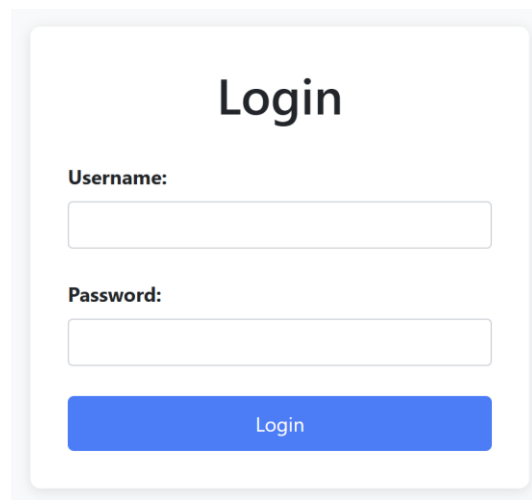
*Figure 1: System Overview*



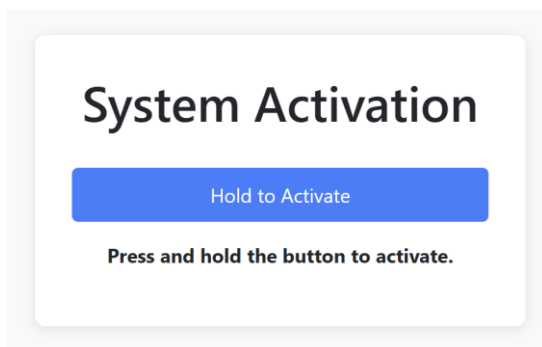
*Figure 2: Communication Overview*

## *Activation Devices*

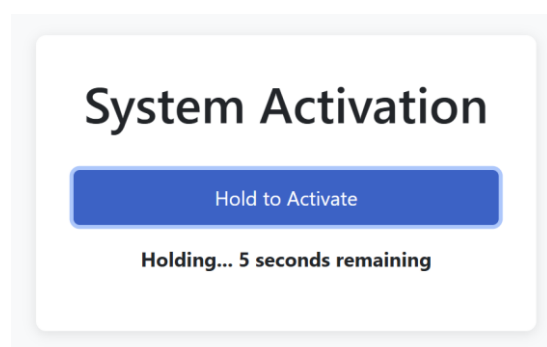
The activation device is designed for simplicity and security. It is hosted on a website by the central server. Users must first login, then hold the activate button for five seconds. A countdown showing time remaining is displayed as the button is held. This two-step process is simple enough to allow for fast activations, but still protects against accidental activations. Once the system is activated, it can only be deactivated by system administrators. The login screen and the activation buttons are shown in *Figure 3* through *Figure 5*.

The image shows a login interface within a light gray rounded rectangle. At the top, the word "Login" is centered in a large, bold, black font. Below it, the label "Username:" is followed by a white rectangular input field. Underneath that, the label "Password:" is followed by another white rectangular input field. At the bottom of the form is a solid blue rectangular button with the word "Login" centered in white text.

*Figure 3: Activation Button Login*

The image shows a "System Activation" prompt interface within a light gray rounded rectangle. The title "System Activation" is centered at the top in a large, bold, black font. Below the title is a solid blue rectangular button with the text "Hold to Activate" centered in white. Underneath the button, the text "Press and hold the button to activate." is displayed in a smaller, bold, black font.

*Figure 4: Activation Prompt*

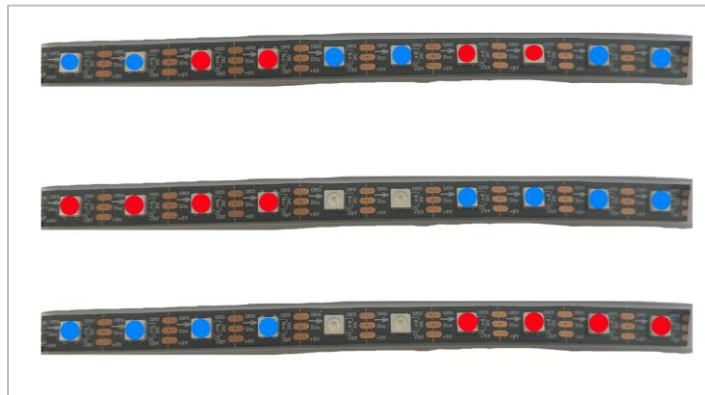
The image shows the "System Activation" prompt interface while the button is being held. It features the same "System Activation" title and blue "Hold to Activate" button as Figure 4. However, the text below the button has changed to "Holding... 5 seconds remaining" in a bold, black font, indicating the active countdown period.

*Figure 5: Activation Prompt while Holding*

## ***Distraction Devices***

The distraction device is hosted on a Raspberry Pi Zero 2W. Upon receiving an activate signal from the central server, it begins looking for the shooter by performing motion detection. Once the shooter is detected, several things happen. First, the device sends a signal back to the central server saying that the shooter has been detected. Second, distractions are deployed. There are three available distractions: lights, noise, and marbles. The distractions are randomly selected and occur at random time intervals. Upon deactivation, the distraction device sends a log file to the central server containing data about which distractions were deployed and when.

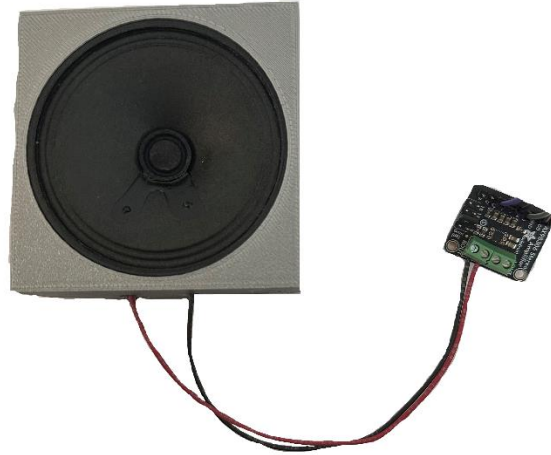
The lights distraction is made from a neopixel strip containing 10 bulbs. The bulbs are programmed to illuminate and flash in a pattern similar to the lights on a police car. This pattern is shown in *Figure 6*.



*Figure 6: Neopixel Light Patterns*

The audio distraction has two options; it can play police sirens or a man yelling. Since the Raspberry Pi Zero does not have a headphone jack, the GPIO pins are used to play audio. First, the Raspberry Pi must be setup properly. The following lines must be added to the `/boot/config.txt` file: `dtoverlay=audremap,pins_12_13` and `dtoverlay=audremap,enable_jack`.

Next, the amplifier is hooked up to the proper GPIO pins. Finally, a .wav file is loaded onto the Raspberry Pi and aplay is used to output the sound file over the GPIO pins.



*Figure 7: Speaker*

The marble distraction spins two horizontal wheels, then releases one marble at a time to be propelled out by the wheels. It works by spinning up two DC motors. Unfortunately, the DC motors alone are not fast enough to shoot the marble out with any force, so gears are used to spin the wheels even faster. The wheels are printed with a small 6-prong gear attached to them, and much larger 24-prong gears are printed to attach to the motors. The wheels are then wrapped in tennis grip tape to allow them to more easily grab onto the marbles. The marbles are stored in a tube. A small servo motor near the bottom of the tube releases and stops the marbles.



*Figure 8: Marble Shooter*

### ***Central Server***

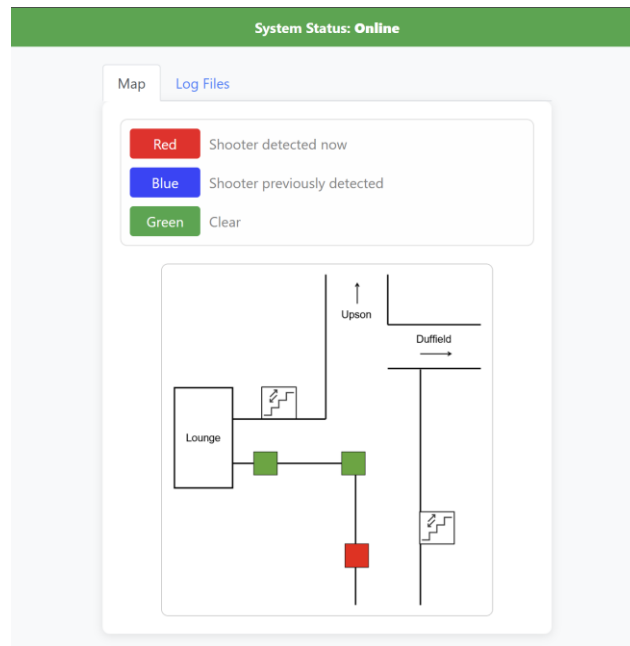
The central server is hosted on a Raspberry Pi 4 Model B. It serves as the communication hub for all devices in the system. It also provides information such as system status, shooter location, and distraction deployment.

The main feature of the central server is a website that displays system information. The server hosts an html webpage that receives data from Flask. This data is updated as messages from the activation fobs and distraction devices come in. It is accessible from a login page identical to that of *Figure 3*.

The system status is shown on the top of the website. It has three states: idle, online, and offline. Idle indicates that the system is online but is not currently doing anything. Online indicates that the system is both activated and online. Offline indicates that certain devices are unreachable.

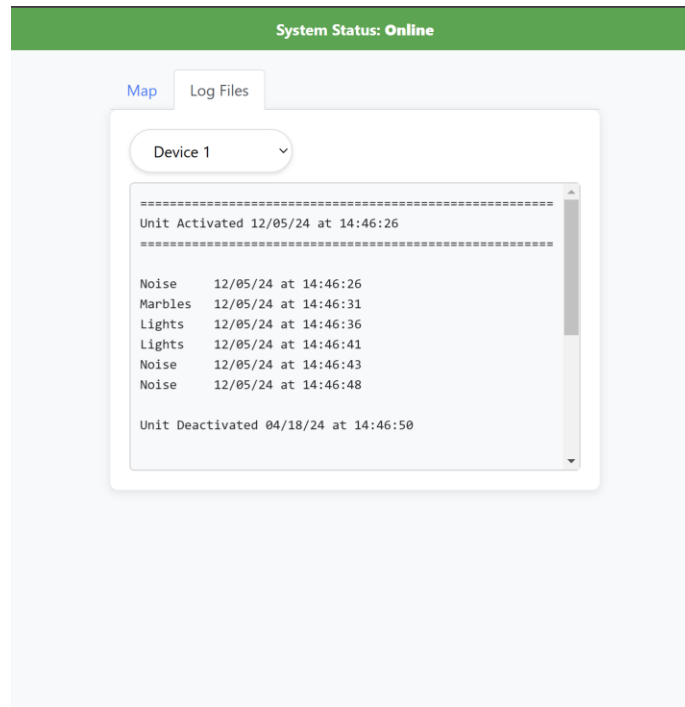
The shooter location is indicated on the map tab of the web interface. If a distraction device has not detected the shooter, the device is colored green. If it has detected the shooter in the past but does not currently see the shooter, the device is colored blue. If the device currently detects the

shooter, it is colored red. This provides law enforcement critical information about the shooter's whereabouts once they arrive on scene, hopefully providing a quicker response. The map is shown in *Figure 9*.



*Figure 9: Central Server Map*

The server also displays distraction log files. These log files are sent back to the central server by each device once the system is deactivated.



*Figure 10: Central Server Log Files*

## ***Communication***

All communication between devices is done via Messaging Queuing Telemetry Transport (MQTT). MQTT is a lightweight, reliable, and secure messaging protocol. It is designed for resource-constrained devices with unstable network environments. Since Raspberry Pi's don't have a lot of computing resources and the environment that these devices will end up in is unknown, this is a perfect protocol for this system.

MQTT offers three Quality of Service (QoS) levels. Level 0, at most once, sends a message once but the receiver does not need to acknowledge it in any way. Level 1, at least once, will periodically send a message until it gets an acknowledgement from the receiver. Level 2, exactly once, requires multiple levels of sending data then receiving acknowledgements. For this system, it only matters that the message was received. A repeat message has no harm on the execution or performance of the system. Therefore, the devices communicate with QoS 1.

Communication between all activation devices and the central server is done to the same topic: act\_dev. To activate the system, the activation device publishes the message shown in *Figure 11*. The server, which is subscribed to this channel, receives the message then responds accordingly.

```
{  
  "device_type": "act",  
  "device_num": <act_num>,  
  "message": "activate"  
}
```

*Figure 11: Activation Signal*

Communication between all distraction devices and the central server is also done to the same topic: dist\_dev. To activate the system, the server simply publishes “activate” to the topic. Upon detecting the shooter, the distraction device publishes a “detected” signal, as shown in *Figure 12*. The log files are sent back to the server in a similar fashion.

```
{  
  "device_type": "dist",  
  "device_num": <dist_num>,  
  "message": "shooter_detected"  
}
```

*Figure 12: Shooter Detected Signal*

## **Results**

The system has been extensively tested. Individual components were tested many times in development. Additionally, the whole system was tested by running through every possible combination of actions. Every state of the website was explored, every distraction device was activated, and every activation fob did the activation. The system works very well.

## Conclusion

The Run, Hide, Activate system aims to provide a robust and effective response to active shooter incidents. The system has multiple ways to distract the shooter in the form of lights, noises, and marbles. It also sends real-time data back to the central server, providing key information to first responders. The Run, Hide, Activate system was tested extensively and performs as expected. It can be expanded by adding a camera for shooter detection, adding more distractions, and adding communication between distraction devices for some cool effects.

## Appendix

All code for this project can be found at <https://github.com/agh93/RunHideActivate>.

## Acknowledgement

Special thank you to Professor Joe Skovira for all his help and guidance with this project.

## References

- <https://learn.adafruit.com/adding-basic-audio-output-to-raspberry-pi-zero?view=all#low-pass-filter-wiring-1960165>
- <https://learn.adafruit.com/stereo-3-7w-class-d-audio-amplifier/overview>
- <https://learn.adafruit.com/neopixels-on-raspberry-pi/python-usage#python-installation-of-neopixel-library-5-1>
- <https://learn.adafruit.com/pir-passive-infrared-proximity-motion-sensor/testing-a-pir>
- <https://raspberrypi.stackexchange.com/questions/133199/no-audio-through-jack-despite-config-audio-on-raspi-config-also-fails>
- <https://flaviocopes.com/python-http-server/>
- <https://stackoverflow.com/questions/75138936/processing-data-from-an-mqtt-stream-using-paho-mqtt>
- <https://www.stlgears.com/generators/3dprint>